PROGRESS

Phase 2 工作汇报：智能化研究框架

> 汇报时间：2026年3月
> 汇报人：[作者]
> 项目：LLM隐私外部性Benchmark框架 — 第二阶段

一、背景与动机

1.1 第一阶段成果回顾

第一阶段（Phase 1）已完成针对KDD/TKDE投稿的全部研究实验：围绕三篇经典数据市场论文（Rhodes & Zhou 2019、Acemoglu et al. 2022、Bergemann & Bonatti 2022）构建了三个可计算的隐私外部性场景（A/B/C），对12款主流LLM进行了系统评估，并建立了"终局偏差 + 机制理解 + 动态收敛"三层诊断指标体系。

1.2 第二阶段的目标

Phase 1 的研究流程高度依赖人工：研究者需要亲自阅读论文、手写场景求解器、设计提示词实验、分析结果。这一流程在面对新论文时无法复用，扩展性差。

Phase 2 的核心目标：将整个研究流程智能化。研究者只需提供一篇论文PDF，系统自动完成场景提取、求解器构建、实验设计与执行、假设验证报告生成。人类研究者的角色从"执行实验"转变为"在三个关键节点把关实验质量"。

二、系统架构设计

2.1 设计原则

在动手实现之前，我们确定了四条核心设计原则：

原则一：Paradigm-as-Contract（范式即契约）
研究者定义"测什么、为什么测"；Agent实现"怎么测"。一份结构化的 `paradigm.yaml` 文件作为研究者意图与Agent行为之间的稳定契约，格式如下：

```yaml
hypotheses:
  - id: H1
    statement: "rho increases → equilibrium share_rate decreases"
    nature: comparative_static        # 决定使用哪种测试类型
```

```
    preferred_test: SensitivityTest
    success_criterion: "EAS < 0 for all rho grid points"
```

原则二：协作网络，非流水线
Agent之间点对点通信，多Agent并行迭代。我们明确拒绝了简单的A→B→C流水线设计，因为它无法支持Agent之间的协商与反馈循环。

原则三：输入即论文PDF
系统从一篇学术论文出发，所有后续工作（场景提取、求解器、实验设计）都由Agent从论文中自动推导。

原则四：Test Registry 可扩展性
测试类型是注册抽象，不是硬编码逻辑。新场景的新假设类型可以触发Agent协作定义新的 `BaseTest` 子类，经人工批准后永久注册。

2.2 整体架构：8个Agent，4层

系统由8个Agent分布在4个层次，共享一个工作空间（`phase2/workspace/`）作为"共享内存"：

```
Layer 1（理解层）: Paper Agent ↔ Scenario Extractor ↔ Solver Builder
                                ↓ Gate 1（人工审核求解器）

Layer 2（设计层）: Market Designer ↔ Orchestrator
                                ↓ Gate 2（人工审核实验设计）

Layer 3（执行层）: Runner ↔ Analysis Agent（动态调整）
                                ↓ 假设PASS/FAIL报告，必要时触发Gate 3

Layer 4（优化层）: Prompt Engineer
                                ↓ Gate 3（人工审核提示词变更）
```

各Agent的核心职责：

Agent	核心问题	主要输出
Paper Agent	这篇论文讲了什么?	`paper_parse.json`
Scenario Extractor	这是什么博弈？H1/H2/H3是什么?	`paradigm.yaml`
Solver Builder	如何计算均衡?	`solver_b.py` + 自验证报告
Market Designer	如何设置模拟市场?	`market_config.yaml`
Orchestrator	现在运行什么实验？覆盖率是否充足?	实验调度决策
Runner	如何可靠地运行LLM实验?	`raw_results/`

Agent	核心问题	主要输出
Analysis Agent	结果意味着什么？假设是否验证？	`hypothesis_status.json` + 图表
Prompt Engineer	为什么提示词失败？如何改进？	新版提示词

2.3 三个人工审核节点（Gate）

Gate	触发时机	人工审核内容
Gate 1	Layer 1三方联合签字	求解器数学正确性、假设可测性
Gate 2	Layer 2设计协商完成	市场角色设置是否忠实于论文模型
Gate 3	Analysis判定prompt质量是瓶颈	新prompt版本不破坏跨版本可比性

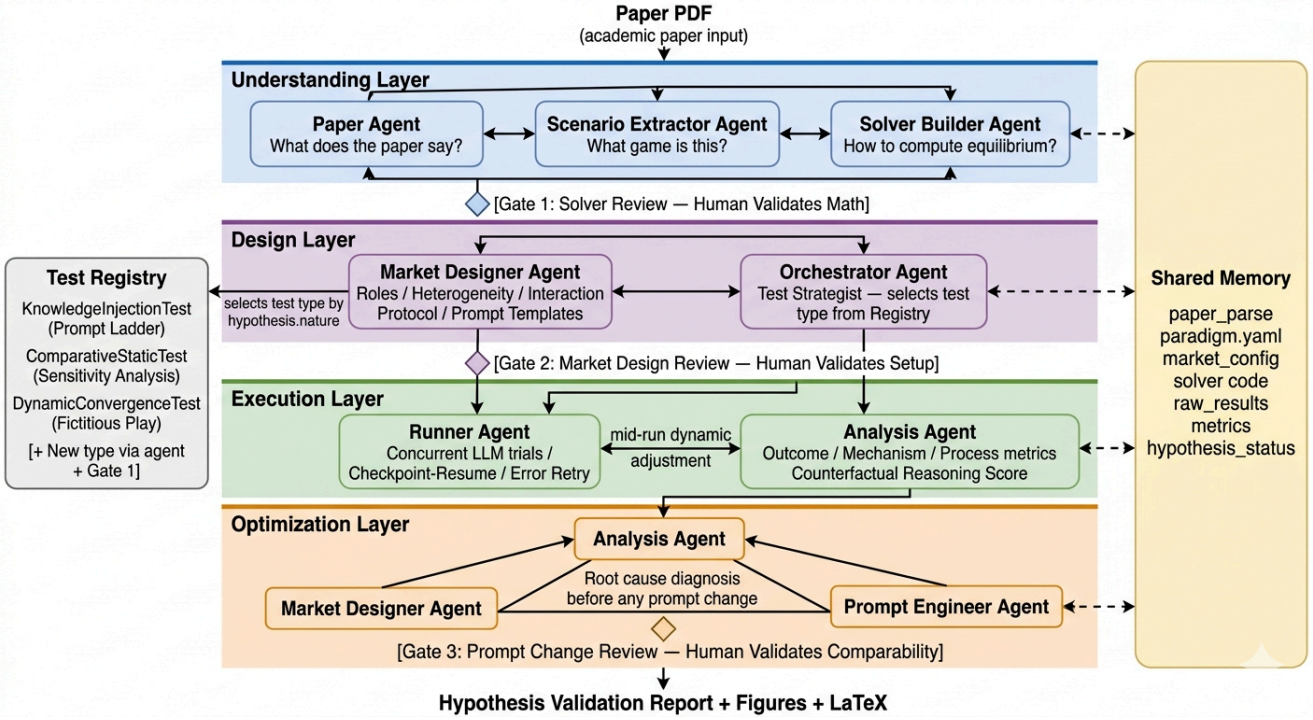
2.4 Test Registry

系统目前注册了三种测试类型，对应场景B的三条假设：

测试类型	假设Nature标签	场景B实例	Phase 1对应
Comparative Static Test	`comparative_static`	3×3参数网格敏感度分析	`run_sensitivity_b.py`
Knowledge Injection Test	`knowledge_dependent`	提示词版本实验（v1→v6）	`run_prompt_experiments.py`
Dynamic Convergence Test	`dynamic_convergence`	虚拟博弈（Fictitious Play）	FP评估模式

2.5 技术选型

维度	选择	理由
Agent框架	自研（Plan A：文件消息队列）	避免引入重型依赖；Plan B（Claude Code multi-agent SDK）作为升级路径
LLM接口	OpenAI-compatible SDK	支持用户自由切换provider（Anthropic / OpenAI / DeepSeek / 本地部署）
Agent间通信	文件消息队列（`workspace/messages/`）	简单可调试；迁移到Plan B时只替换路由机制，业务逻辑不变
共享内存	文件系统（`workspace/`）	天然持久化，支持断点续跑



三、当前实现进展

3.1 工具层（`tools/`）— 已完成并测试

工具层是所有Agent的基础设施，已经过完整的单元测试：

`file_io.py`：workspace统一读写接口，包含JSON/YAML/文本读写、消息队列（send/list/mark）和Gate辅助函数。所有操作通过相对路径访问workspace，Agent不直接操作文件系统。

`agent_api_client.py`：统一LLM客户端，支持两种provider：

- Anthropic原生SDK：处理 `tool_use` 响应块、 `tool_result` 消息格式
- OpenAI-compatible SDK：处理 `tool_calls` 、消息对象序列化、 `arguments` JSON解析

两种provider使用相同的tool spec格式（`make_tool_spec`），内部自动转换。支持指数退避重试、rate limit处理和快速失败（`max_rate_limit_retries` 参数）。

用户通过 `phase2/config.json` 配置：

```
{
  "agent_llm": {
    "provider": "openai",
    "model": "gpt-5.2",
    "api_key": "...",
    "base_url": "https://..."
  }
}
```

`pdf_reader.py`：基于pdfplumber的PDF文字提取（非OCR，直接解析PDF字符流），支持启发式section分割和关键词搜索。

3.2 Layer 1 — 已完成并端到端测试通过

Paper Agent：读取PDF，使用tool-use循环让Claude调用 `search_paper`、`get_section`、`write_paper_parse` 工具，最终产出结构化的 `paper_parse.json`（包含各section提取文本、关键变量、支付函数、核心机制描述）。

Scenario Extractor：读取 `paper_parse.json`，提取博弈结构并形式化三条假设，写入 `paradigm.yaml`。可向Paper Agent发起查询，向Solver Builder发送假设草稿请求验证。

Solver Builder：读取 `paper_parse.json` 和 `paradigm.yaml`，写Python求解器代码，通过 `subprocess` 执行三项自验证（边界条件、方向性、合理性检验）。

协调器（`layer1_coordinator.py`）：串行运行三个Agent（Paper → Extractor → Solver Builder），主线程负责路由发往Paper Agent的消息，Extractor和Solver Builder在各自线程中运行并等待回复。Gate 1触发后等待人工写入 `gate1_approved.json`。

端到端测试结果（以Acemoglu 2022 "Too Much Data"为例）：

```
Phase A: paper_parse.json 写入（5个顶层字段）          ✅
Phase B-1: paradigm.yaml 写入，3条假设结构完整          ✅
Phase B-2: solver_b.py 写入，3项自验证全部通过          ✅
Gate 1: 三方签字，人工审核通过                          ✅
总耗时: ~2分钟（主要为API响应时间）
```

四、遇到的主要问题与解决方案

4.1 Rate Limit 问题

现象：多个Agent并发调用同一API端点时，频繁触发429 Too Many Requests。即使改为串行，Extractor用完配额后Solver Builder启动时仍被限速。

临时方案：

- Layer 1改为严格串行执行
- Solver Builder使用 `max_rate_limit_retries=1`，首次rate limit后立即放弃API调用，转为fallback

根本方案（待实现）：API池——多个key轮换，某个key触发429时立即切换到下一个。

4.2 论文内容被误判为Prompt Injection

现象：将论文原文（含希腊字母ρ、数学公式、JSON结构等特殊字符）拼入user message时，Claude的安全机制将其误判为prompt injection攻击，拒绝执行并跳出角色（"I'm Claude,

an AI assistant...")。

修复：所有论文内容用XML标签包裹，明确标注为数据而非指令：

```
<paper_content>
{论文原文}
</paper_content>
```

4.3 消息路由竞态

现象：协调器主线程调用 `extractor.handle_pending_messages()` 时，将Paper Agent发给Extractor的回复标记为 `done`；Extractor线程的 `_wait_for_reply` 只查询 `status=pending`，因此永远找不到回复，120秒后超时。

修复：`_wait_for_reply` 改为按 `reply_to` 消息ID扫描全部消息（不依赖status）；协调器的消息路由循环只处理发往Paper Agent的消息，其他Agent自行管理各自收件箱。

4.4 Agent核心功能依赖Fallback（待解决）

现象：Extractor和Solver Builder的输出实际来自硬编码的fallback，而非Claude真正从论文推导的结果：

- Extractor：Claude完成了多轮API调用，但未调用 `write_paradigm` 工具，触发 `_write_default_paradigm()`
- Solver Builder：因rate limit，API调用失败，触发 `_write_fallback_solver()`

影响：当前系统对新论文不具备通用性。换一篇论文，输出仍是场景B的硬编码内容。这是Phase 2目前最核心的待解决问题。

计划：API池解决rate limit后，重新审视Extractor的prompt设计，确保Claude真正调用工具产出结果，并收紧fallback触发条件。

五、下一步计划

阶段 A：基础修复（最高优先级）

解决Agent"名义运行、实质fallback"的问题，是后续一切工作的前提。

A1. API池

支持多key轮换，从根本上消除rate limit对Agent并发的阻断。实现后可恢复并行执行，也为Layer 3大规模实验提供基础。

A2. Extractor真正智能化

- 加日志区分"Claude调用工具"与"fallback触发"

- 改进prompt，明确要求Claude的最终动作必须是调用 `write_paradigm`
- 验收标准：输入新论文时， `paradigm.yaml` 内容与该论文的实际博弈结构一致

A3. Solver Builder真正智能化

API池到位后，测试Claude真正生成求解器代码的能力，并验证自验证三项测试能区分正确与错误的实现。

阶段 B：Test Registry

封装Phase 1的实验代码为标准化测试接口，为Layer 3的自动化执行做准备：

```
class BaseTest:
    def setup(self, params) -> TestEnv: ...
    def run(self, env, model) -> RawResult: ...
    def evaluate(self, raw, ground_truth) -> Metrics: ...
    def verify_hypothesis(self, metrics, hypothesis) -> HypothesisResult:
...
```

三种测试类型（SensitivityTest、PromptLadderTest、FictitiousPlayTest）分别复用Phase 1对应代码。

阶段 C/D：Layer 2 + Layer 3

Layer 2（Market Designer + Orchestrator）协商实验参数，产出 `market_config.yaml` ，通过Gate 2。
Layer 3（Runner + Analysis）执行实验，调用Phase 1的被测模型评估代码，产出假设验证报告。Layer 3是整个Phase 2产生实际研究价值的核心。

阶段 E：Layer 4（按需）

Prompt Engineer仅在Analysis判定prompt质量是瓶颈时触发，通过Gate 3后进入角色提示词库。

六、当前文件状态

```
phase2/
├── DESIGN.md                    完整架构设计文档
├── PROGRESS.md                  本汇报文档
├── config.example.json          用户API配置模板
├── requirements.txt             依赖声明
├── layer1_coordinator.py        Layer 1入口，端到端测试通过
│
├── workspace/
│   ├── input/paper.pdf          Acemoglu 2022（已放入）
```

```
│   ├── paper_parse.json          ✅ 已生成
│   ├── paradigm.yaml             ✅ 已生成（内容正确，fallback路径）
│   ├── solver/solver_b.py        ✅ 已生成（自验证通过，fallback路径）
│   └── gate1_approved.json       ✅ 已人工审核通过
│
├── agents/
│   ├── paper_agent.py            ✅ 已实现，测试通过
│   ├── scenario_extractor.py     ✅ 已实现，测试通过（fallback路径）
│   ├── solver_builder.py         ✅ 已实现，测试通过（fallback路径）
│   ├── market_designer.py       ☐ 未开始（阶段C）
│   ├── orchestrator.py          ☐ 未开始（阶段C）
│   ├── runner.py                ☐ 未开始（阶段D）
│   ├── analysis.py              ☐ 未开始（阶段D）
│   └── prompt_engineer.py       ☐ 未开始（阶段E）
│
├── tools/
│   ├── file_io.py                ✅ 已实现，单元测试通过
│   ├── agent_api_client.py       ✅ 已实现，测试通过
│   └── pdf_reader.py             ✅ 已实现，测试通过
│
└── tests/
    ├── base_test.py             ☐ 未开始（阶段B）
    ├── sensitivity_test.py      ☐ 未开始（阶段B）
    ├── prompt_ladder_test.py    ☐ 未开始（阶段B）
    └── fictitious_play_test.py ☐ 未开始（阶段B）
```